
Algorithm 1 Compute inference model loss

```

fit prior  $p(\mathbf{P})$  on  $\mathbf{P}_1, \dots, \mathbf{P}_k$ 
 $\mathbf{P} \sim p(\mathbf{P})$ 
 $\mathbf{w} \sim p(\mathbf{w}|\mathbf{P})$ 
 $\mathbf{y} \leftarrow c(\mathbf{w})$ 
return  $\phi + \alpha \nabla_{\phi} \log q_{\phi}(\mathbf{y}|\mathbf{P})$ 

```

Algorithm 2 A-NESI training loop

```

Input: dataset  $\mathcal{D}$ , params  $\theta$ , params  $\phi$ 
beliefs  $\leftarrow []$ 
while not converged do
   $(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}$ 
   $\mathbf{P} \leftarrow f_{\theta}(\mathbf{x})$ 
  update beliefs with  $\mathbf{P}$ 
   $\phi \leftarrow \text{Algorithm 1}(\text{beliefs}, \phi)$ 
   $\theta \leftarrow \theta + \alpha \nabla_{\theta} \log q_{\phi}(\mathbf{y}|\mathbf{P})$ 

```

Figure 2: The training loop of A-NESI.

This factorization makes the inference model highly flexible. We can use simpler models if we know the dependencies between variables in W and Y . The factorization is computationally linear in $k_W + k_Y$, instead of exponential in k_W for exact inference. During testing, we use a beam search to find the most likely prediction from $q_{\phi, \psi}(\mathbf{w}, \mathbf{y}|\mathbf{P})$. If the method successfully trained the perception model, then its entropy is low and a beam search should easily find the most likely prediction [37].

There are no restrictions on the architecture of the inference model q_{ϕ} . Any neural network with appropriate inductive biases and parameter sharing to speed up training can be chosen. For instance, CNNs over grids of variables, graph neural networks [27, 48] for reasoning problems on graphs, or transformers for sets or sequences [55].

We use the prediction model to train the perception model f_{θ} given a dataset $\mathcal{D}$ of tuples $(\mathbf{x}, \mathbf{y})$. Our novel loss function trains the perception model by backpropagating through the prediction model:

$$\mathcal{L}_{\text{Perception}}(\mathcal{D}, \theta) = -\log q_{\phi}(\mathbf{y}|\mathbf{P} = f_{\theta}(\mathbf{x})), \quad \mathbf{x}, \mathbf{y} \sim \mathcal{D} \quad (4)$$

The gradients of this loss are biased due to the error in the approximation q_{ϕ} , but it has no variance outside of sampling from the training dataset.

3.2 Training the inference model

We define two variations of our method. The *prediction-only* variant (Section 3.2.1) uses only the prediction model $q_{\phi}(\mathbf{y}|\mathbf{P})$, while the *explainable* variant (Section 3.2.4) also uses the explanation model $q_{\psi}(\mathbf{w}|\mathbf{y}, \mathbf{P})$.

Efficient training of the inference model relies on two design decisions. The first is a descriptive factorization of the output space Y , which we discuss in Section 3.2.2. The second is using an informative *belief prior* $p(\mathbf{P})$, which we discuss in Section 3.2.3.

We first define the forward process that uses the symbolic function c to generate training data:

$$p(\mathbf{w}, \mathbf{y}|\mathbf{P}) = p(\mathbf{w}|\mathbf{P})p(\mathbf{y}|\mathbf{w}, \mathbf{P}) = p(\mathbf{w}|\mathbf{P})\mathbb{1}_{c(\mathbf{w})=\mathbf{y}} \quad (5)$$

We take some example world $\mathbf{w}$ and deterministically compute the output of the symbolic function $c(\mathbf{w})$. Then, we compute whether the output $c(\mathbf{w})$ equals $\mathbf{y}$. Therefore, $p(\mathbf{w}, \mathbf{y}|\mathbf{P})$ is 0 if $c(\mathbf{w}) \neq \mathbf{y}$ (that is, $\mathbf{w}$ is not a possible world of $\mathbf{y}$).

The belief prior $p(\mathbf{P})$ allows us to generate beliefs $\mathbf{P}$ for the forward process. That is, we generate training data for the inference model using

$$p(\mathbf{P}, \mathbf{w}) = p(\mathbf{P})p(\mathbf{w}|\mathbf{P}) \quad (6)$$

where $\mathbf{P}, \mathbf{w} \sim p(\mathbf{P}, \mathbf{w}), \quad \mathbf{y} = c(\mathbf{w})$.

The belief prior allows us to train the inference model with synthetic data: The prior and the forward process define everything q_{ϕ} needs to learn. Note that the sampling process $\mathbf{P}, \mathbf{w} \sim p(\mathbf{P}, \mathbf{w})$ is fast and parallelisable. It involves sampling from a Dirichlet distribution, and then sampling from a categorical distribution for each dimension of $\mathbf{w}$ based on the sampled parameters $\mathbf{P}$.

3.2.1 Training the prediction-only variant

In the prediction-only variant, we only train the prediction model $q_{\phi}(\mathbf{y}|\mathbf{P})$. We use the samples generated by the process in Equation 6. We minimize the expected cross entropy between $p(\mathbf{y}|\mathbf{P})$

and $q_\phi(\mathbf{y}|\mathbf{P})$ over the prior $p(\mathbf{P})$:

$$\mathcal{L}_{Pred}(\phi) = -\log q_\phi(c(\mathbf{w})|\mathbf{P}), \quad \mathbf{P}, \mathbf{w} \sim p(\mathbf{P}, \mathbf{w}) \quad (7)$$

See A for a derivation. In the loss function defined in Equation 7, we estimate the expected cross entropy using samples from $p(\mathbf{P}, \mathbf{w})$. We use the sampled world $\mathbf{w}$ to compute the output $\mathbf{y} = c(\mathbf{w})$ and increase its probability under q_ϕ . Importantly, we do not need to use any data to evaluate this loss function. We give pseudocode for the full training loop in Figure 2.

3.2.2 Output space factorization

The factorization of the output space Y introduced in Section 2 is one of the key ideas that allow efficient learning in A-NESI. We will illustrate this with MNISTAdd. As the number of digits N increases, the number of possible outputs (i.e., sums) is $2 \cdot 10^N - 1$. Without factorization, we would need an exponentially large output layer. We solve this by predicting the individual digits of the output so that we need only $N \cdot 10 + 2$ outputs similar to [3]. Furthermore, recognizing a single digit of the sum is easier than recognizing the entire sum: for its rightmost digit, only the rightmost digits of the input are relevant.

Choosing the right factorization is crucial when applying A-NESI. A general approach is to take the CNF of the symbolic function and predict each clause’s truth value. However, this requires grounding the formula, which can be exponential. Another option is to predict for what objects a universally quantified formula holds, which would be linear in the number of objects.

3.2.3 Belief prior design

How should we choose the $\mathbf{P}$ s for which we train $q_{\phi,\psi}$? A naive method would use the perception model f_θ , sample some training data $\mathbf{x}_1, \dots, \mathbf{x}_k \sim \mathcal{D}$ and train the inference model over $\mathbf{P}_1 = f_\theta(\mathbf{x}_1), \dots, \mathbf{P}_k = f_\theta(\mathbf{x}_k)$. However, this means the inference model is only trained on those $\mathbf{P}$ occurring in the training data. Again, consider the Multi-digit MNISTAdd problem. For $N = 15$, we have a training set of 2000 sums, while there are $2 \cdot 10^{15} - 1$ possible sums. By simulating many beliefs, the inference model sees a much richer set of inputs and outputs, allowing it to generalize.

A better approach is to fit a Dirichlet prior $p(\mathbf{P})$ on $\mathbf{P}_1, \dots, \mathbf{P}_k$ that covers all possible combinations of numbers. We choose a Dirichlet prior since it is conjugate to the discrete distributions. For details, see Appendix F. During hyperparameter tuning, we found that the prior needs to be high entropy to prevent the inference model from ignoring the inputs $\mathbf{P}$. Therefore, we regularize the prior with an additional term encouraging high-entropy Dirichlet distributions.

3.2.4 Training the explainable variant

The explainable variant uses both the prediction model $q_\phi(\mathbf{y}|\mathbf{P})$ and the optional explanation model $q_\psi(\mathbf{w}|\mathbf{y}, \mathbf{P})$. When training the explainable variant, we use the idea that both factorizations of the joint should have the same probability mass, that is, $p(\mathbf{w}, \mathbf{y}|\mathbf{P}) = q_{\phi,\psi}(\mathbf{w}, \mathbf{y}|\mathbf{P})$. To this end, we use a novel *joint matching* loss inspired by the theory of GFlowNets [9, 10], in particular, the trajectory balance loss introduced by [33] which is related to variational inference [34]. For an in-depth discussion, see Appendix E. The joint matching loss is essentially a regression of $q_{\phi,\psi}$ onto the true joint p that we compute in closed form:

$$\mathcal{L}_{Expl}(\phi, \psi) = \left(\log \frac{q_{\phi,\psi}(\mathbf{w}, c(\mathbf{w})|\mathbf{P})}{p(\mathbf{w}|\mathbf{P})} \right)^2, \quad \mathbf{P}, \mathbf{w} \sim p(\mathbf{P}, \mathbf{w}) \quad (8)$$

Here we use that $p(\mathbf{w}, c(\mathbf{w})|\mathbf{P}) = p(\mathbf{w}|\mathbf{P})$ since $c(\mathbf{w})$ is deterministic. Like when training the prediction-only variant, we sample a random belief $\mathbf{P}$ and world $\mathbf{w}$ and compute the output $\mathbf{y}$. Then we minimize the loss function to *match* the joints p and $q_{\phi,\psi}$. We further motivate the use of this loss in Appendix D. Instead of a classification loss like cross-entropy, the joint matching loss ensures $q_{\phi,\psi}$ does not become overly confident in a single prediction and allows spreading probability mass easier.

3.3 Symbolic pruner

An attractive option is to use symbolic knowledge to ensure the inference model only generates valid outputs. We can compute each factor $q_\psi(w_i|\mathbf{y}, \mathbf{w}_{1:i-1}, \mathbf{P})$ (both for world and output variables)

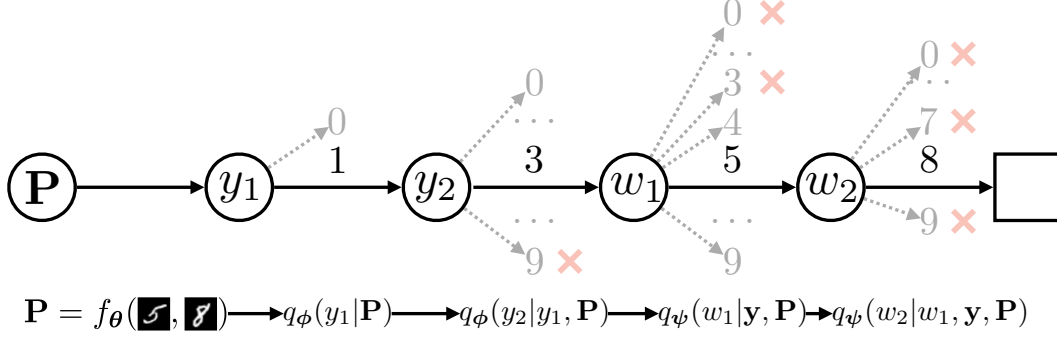


Figure 3: Example rollout sampling an explainable variant on input $\mathbf{x} = (\mathbf{5}, \mathbf{8})$. For y_2 , we prune option 9, as the highest attainable sum is $9 + 9 = 18$. For w_1 , $\{0, \dots, 3\}$ are pruned as there is no second digit to complete the sum. w_2 is deterministic given w_1 , and prunes all branches but 8.

using a neural network $\hat{q}_{\psi,i}$ and a *symbolic pruner* $s_{\mathbf{y}, \mathbf{w}_{1:i-1}} : W_i \rightarrow \{0, 1\}$:

$$q_{\psi}(w_i = j|\mathbf{y}, \mathbf{w}_{1:i-1}, \mathbf{P}) = \frac{\hat{q}_{\psi}(w_i = j|\mathbf{y}, \mathbf{w}_{1:i-1}, \mathbf{P})s_{\mathbf{y}, \mathbf{w}_{1:i-1}}(j)}{\sum_{j' \in W_i} \hat{q}_{\psi}(w_i = j'|\mathbf{y}, \mathbf{w}_{1:i-1}, \mathbf{P})s_{\mathbf{y}, \mathbf{w}_{1:i-1}}(j')} \quad (9)$$

The symbolic pruner sets the probability mass of certain choices for the variable w_i to zero. Then, q_{ψ} is computed by renormalizing. If we know that by expanding $\mathbf{w}_{1:i-1}$ it will be impossible to produce a possible world for $\mathbf{y}$, we can set the probability mass under that branch to 0: we will know that $p(\mathbf{w}, \mathbf{y}) = 0$ for all such branches. In Figure 3 we give an example for single-digit MNISTAdd. Symbolic pruning significantly reduces the number of branches our algorithm needs to explore during training. Moreover, symbolic pruning is critical in settings where verifiability and safety play crucial roles, such as medicine. We discuss the design of symbolic pruners s in Appendix G.

4 Experiments

We study three Neurosymbolic reasoning tasks to evaluate the performance and scalability of A-NeSI: Multi-digit MNISTAdd [37], Visual Sudoku Puzzle Classification [4] and Warcraft path planning. Code is available at <https://github.com/HEmile/a-nesi>. We used the ADAM optimizer throughout.

A-NeSI has two prediction methods. 1) **Symbolic prediction** uses the symbolic reasoning function c to compute the output: $\hat{\mathbf{y}} = c(\arg \max_{\mathbf{w}} p(\mathbf{w}|\mathbf{P} = f_{\theta}(\mathbf{x})))$. 2) **Neural prediction** predicts with the prediction network q_{ϕ} using a beam search: $\hat{\mathbf{y}} = \arg \max_{\mathbf{y}} q_{\phi}(\mathbf{y}|\mathbf{P} = f_{\theta}(\mathbf{x}))$. In our studied tasks, neural prediction cannot perform better than symbolic prediction. However, we still use the prediction model to efficiently train the perception model in the symbolic prediction setting. We consider the prediction network adequately trained if it matches the accuracy of symbolic prediction.

4.1 Multi-digit MNISTAdd

We evaluate A-NeSI on the Multi-Digit MNISTAdd task (Section 2). For the perception model, we use the same CNN as in DeepProbLog [35]. The prediction model has $N+1$ factors $q_{\phi}(y_i|\mathbf{y}_{1:i-1}, \mathbf{P})$, while the explanation model has $2N$ factors $q_{\psi}(w_i|\mathbf{y}, \mathbf{w}_{1:i-1}, \mathbf{P})$. We model each factor with a separate MLP. y_i and w_i are one-hot encoded digits, except for the first output digit y_1 : it can only be 0 or 1. We used a shared set of hyperparameters for all N . For more details and a description of the baselines, see Appendix I.1.

Table 1 reports the accuracy of predicting the sum. For all N , A-NeSI is close to the reference accuracy, meaning there is no significant drop in the accuracy of the perception model as N increases. For small N , it is slightly outperformed by DeepStochLog [56], which can not scale to $N = 15$. A-NeSI also performs slightly better than DeepProbLog, showing approximate inference does not hurt the accuracy. With neural prediction, we get the same accuracy for low N , but there is a significant drop for $N = 15$, meaning the prediction network did not perfectly learn the problem. However, compared to training a neural network without background knowledge (Embed2Sym with